

当前架构与实现说明

面向多用户、分布式 Agent 云服务的 PG-first 运行时框架

本文档基于当前代码库与线上部署形态整理，描述实际已经存在的组件、数据流、职责边界、扩展方式和已知边界。它不是理想化设计稿，而是当前实现的架构基线。



版本：PG-first distributed runtime

更新时间：2026 年 8 月

01 设计定位与核心原则

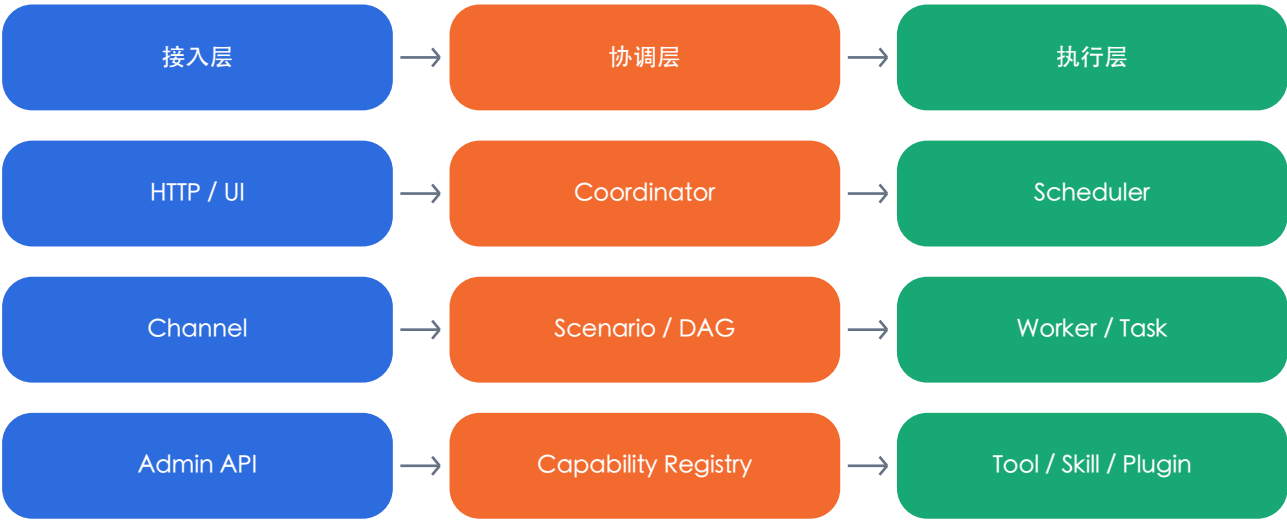
Joyhousebot 已从单机 Agent 形态收敛为一个可承载多个用户、多个会话、多个并发任务的 Agent 服务框架。核心不是某一个具体业务 Agent，而是把请求接入、意图协调、能力调用、异步执行、状态持久化和全过程追踪标准化。

原则	当前实现
PG-first	PostgreSQL 是 Run、Task、事件、Trace、记忆、租约和 Channel Outbox 的共享权威。
单一运行模型	HTTP、Web UI、Channel 最终都转换为统一 Run，不维护两套 Agent 执行链。
主 Agent 协调	Coordinator 负责意图、场景、追问和计划；具体业务能力由 Tool、Skill、Plugin 承担。
可解释执行	阶段事件、决策摘要、工具调用、耗时和产物均可查询与回放。
业务解耦	dinq-discover 作为 dinq-plugin 接入，不把业务代码写入 Joyhousebot 核心。

当前系统的主要取舍

- 优先保证可恢复、可观测和可扩展，再优化单次请求的最低延迟。
- 所有长任务使用持久化 Run，短任务也沿用同一模型，以降低系统复杂度。
- 模型原始隐藏推理不作为公共接口契约；对外展示结构化思考摘要和执行事件。

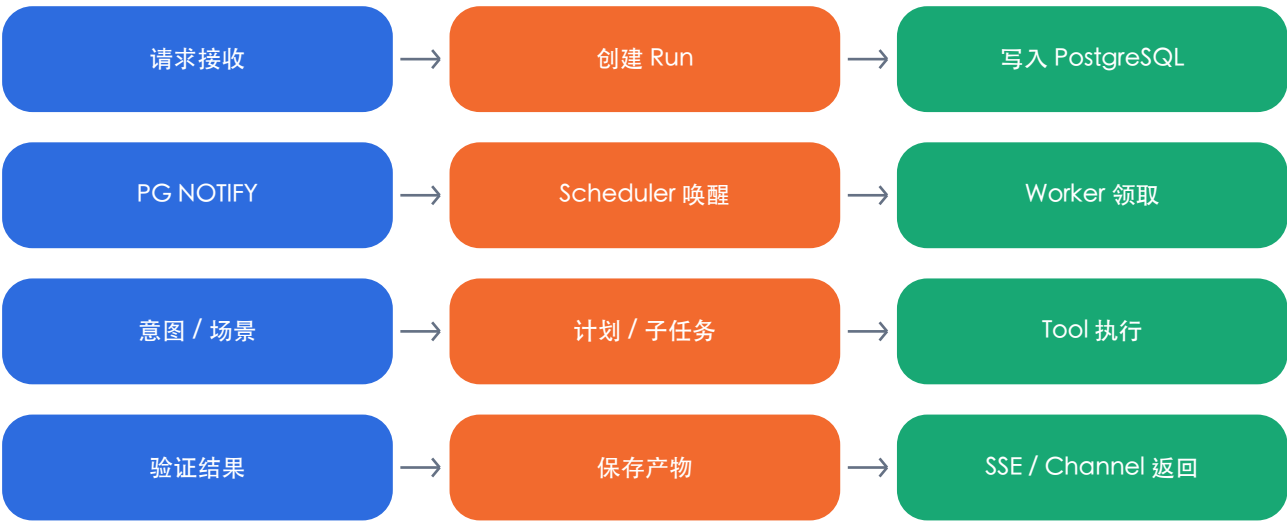
02 总体分层与组件职责



层	职责	主要代码区域
接入层	用户认证、会话、Run 提交、SSE、Channel 消息接入。	joyhousebot/api、joyhousebot/channels
协调层	意图识别、场景匹配、追问判断、能力选择、执行计划。	joyhousebot/runtime/request_coordination.py
调度层	入队、领取、Lease、重试、超时、恢复、通知唤醒。	joyhousebot/runtime、joyhousebot/storage
能力层	工具、Skill、Scenario、Plugin 的发现、校验、调用和结果规范化。	joyhousebot/capabilities、ding-plugin
状态层	PostgreSQL 表、事件、Trace Blob、Memory、Outbox。	joyhousebot/storage、joyhousebot/agent
控制台	运行监控、配置、Agent 试用、场景模拟和插件运维。	frontend/

03 一次请求的完整执行链

每条用户消息都以 user_id、session_id 和 agent_id 作为基础身份上下文，创建一个可持久化 Run。短请求和长任务使用相同生命周期，区别只在执行时长、超时策略和交付方式。



调度细节

- SKIP LOCKED 是唯一任务竞争裁决；PG NOTIFY 只负责低延迟唤醒，不负责一致性。
- 正常空闲 Worker 时，入队到领取目标是几十毫秒级；轮询仅作为 100-250ms 可配置的容灾兜底。
- 事件中记录 wake_source、queue_wait_ms、claim_latency_ms，可区分通知漏唤醒、无 Worker 容量和数据库延迟。
- Worker 使用租约和心跳执行任务，异常退出后由恢复流程重新开放或标记失败。

Run 状态示例

状态	含义	下一步
queued	已接受并等待执行	Scheduler / Worker 领取
running	正在协调或执行 Task	继续执行、重试或等待子任务
waiting_input	需要用户补充场景要求	用户回复后恢复同一会话
completed / failed	已完成或不可恢复失败	展示结果、事件和诊断

04 Coordinator、场景与追问 DAG

Coordinator 是对外服务的主 Agent，负责把自然语言请求转换为可验证、可执行的计划。它不直接承载 dinq-discover 的全部业务逻辑，而是消费已注册的能力。



场景配置包含

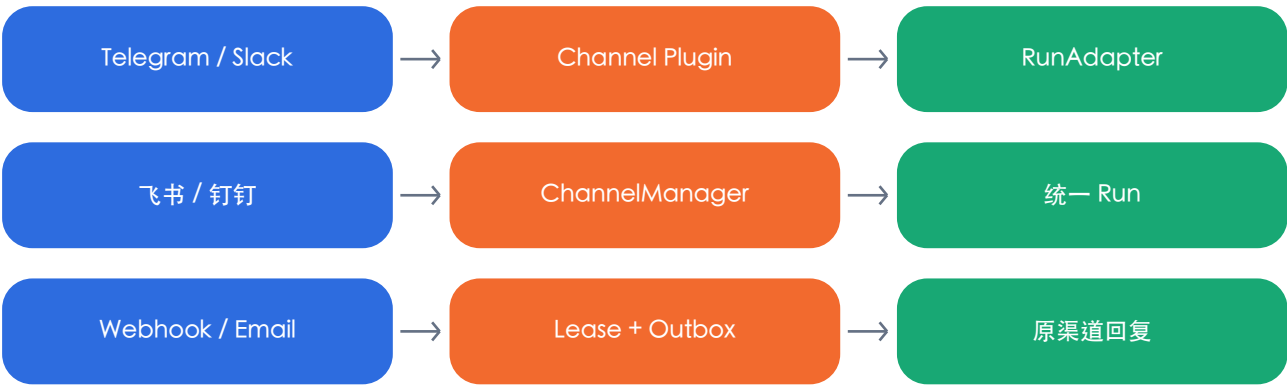
- 场景 ID、名称、版本、说明和启用状态。
- 触发条件、必填字段、字段类型和默认值。
- 追问节点、问题顺序、跳转条件和最大追问次数。
- 允许的 Tool、Skill、Plugin、并发策略和结果规范。
- 模拟输入、预期路由和测试结果，供业务人员在控制台验证。

以 dinq-discover 为例

推荐保留一个 Main Coordinator。它负责判断是人才搜索、候选人丰富、平台搜索还是通用问答，再调用 dinq-plugin 中的搜索工具和 Skill。场景命中后，Coordinator 可以先补充地点、岗位、经验等必要条件，再启动并发搜索和结果合并。

05 Channel 外部消息接入层

Channel 是消息系统适配层，不是另一套 Agent Runtime。所有 Channel 消息都会被规范化为统一的 user_id、session_id、agent_id 和 Run。



组件	作用
Channel Plugin	实现外部平台的连接、消息解析、发送和平台特有功能。
ChannelManager	加载启用的渠道，维护连接、Lease 和发送 Worker。
ChannelRuntimeBridge	将进站消息转换为 Run，并将终态结果路由回原消息。
channel_outbox	持久化出站消息，支持重试、幂等和多进程投递。
RunAdapter	定义 Channel 与 Runtime 之间的统一适配契约。

当前状态

Channel 是可选能力。配置为空时 API 和 Web UI 仍可正常工作，日志中的 No channels enabled 只表示没有启动外部消息连接器。当前适配器仍随核心包内置，但 ChannelPlugin、RunAdapter 和 ChannelRuntimeBridge 已形成独立边界；未来可拆成 joyhousebot-channel-* 包，不改变统一 Run/Task 契约。

06 Memory 记忆层

Memory 是可按 Agent 配置启用的用户个性化能力。运行时以 PostgreSQL 为唯一权威，MEMORY.md 等名称只是逻辑文档路径和导入导出格式，不再依赖本地文件。



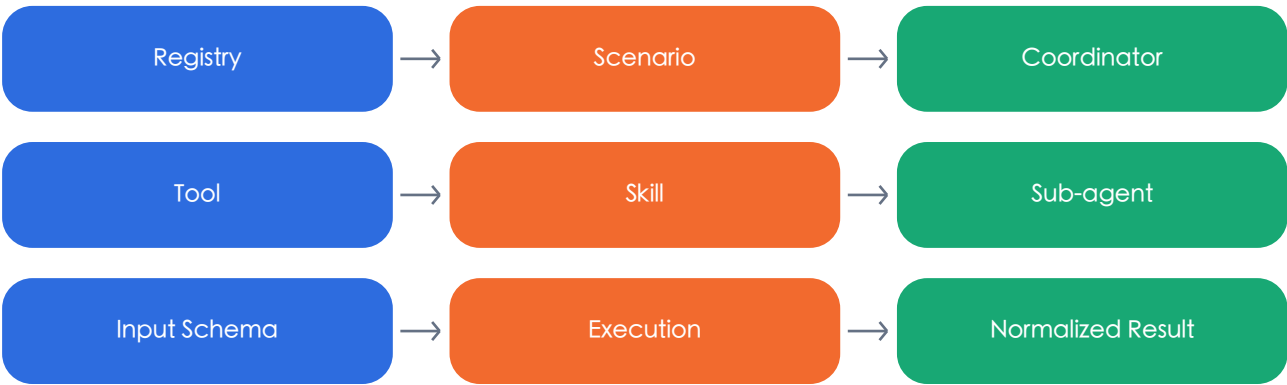
记忆层	用途	默认特点
working	当前执行过程临时上下文	读，不持久化
session	当前会话上下文	随会话持久化
episodic	历史事件和每日记录	可追加、可裁剪
profile	用户个人属性和偏好	按用户隔离
long_term	稳定事实和长期知识	按用户和 Agent 隔离
agent	Agent 自身知识和配置	默认不向用户上下文暴露

dinq-discover 建议

搜索型 Agent 通常关闭 profile 和 long_term，避免把个人历史带入候选人搜索；若需要保存用户常用搜索条件，可以只启用 session 或受控 profile。

07 Capability Plugin、Tool 与 Skill

能力注册表把框架执行机制与业务实现隔离。Joyhousebot 只负责发现、授权、调用、超时、重试、记录和结果规范化；dinq-discover 的搜索逻辑放在独立 dinq-plugin 中。



能力类型	定位	示例
Tool	一次明确的外部操作，具有输入输出契约。	dinq.talent.filter
Skill	可复用的业务策略、提示和工具组合。	按条件找人、候选人丰富
Scenario	定义路由、追问和能力边界。	Dinq Discover Search
Plugin	独立分发的一组工具、Skill、Scenario 和适配器。	dinq-plugin

统一调用契约

调用前校验输入 Schema 和能力边界，调用中记录准备、开始、成功或失败事件，调用后将供应商或业务返回转换为统一结果结构，并保留原始诊断信息供回放。

MCP 对外适配

Streamable HTTP MCP 网关位于 /mcp/。已发布且启用的 tool / connector 能力会动态映射为 MCP tools；tools/call 不直接执行业务函数，而是创建持久化 Run/Task，复用同一套鉴权、权限、Lease、事件、Trace、产物和回放链路。MCP 是协议适配层，不是第二套执行运行时。

08 可观测性、Trace 与前端控制台

系统把一次 Run 看成可回放的执行事实，而不是只有最终文本。控制台因此可以同时承担 Agent 试用、运行监控、配置管理和插件运维。



配置入口边界

侧栏将平台收拢到配置菜单的子项。平台只负责访问控制、集群发布、审计和运行摘要；Agent、Skills、Tools、MCP Server 分别在配置子菜单维护；Dinq 运维作为独立插件运维入口保留，避免平台页与能力编辑页重复。

前端可以观察的关键事实

- 请求接受、进入队列、调度决策、Worker 领取、阶段切换。
- Coordinator 的意图、场景、能力选择和追问摘要。
- 模型调用耗时、工具准备与执行耗时、Task 并发进度。
- 任务失败原因、重试次数、供应商错误、产物保存和验证结果。
- PG NOTIFY、poll、recovery 唤醒来源及 queue_wait_ms、claim_latency_ms。

显示边界

对外展示结构化决策摘要和事件，不把模型供应商的隐藏原始思维链作为稳定 API 契约。开发调试环境可以保留更细的内部 Trace，但需要权限控制、脱敏和保留期限。

09 PostgreSQL 数据与分布式一致性

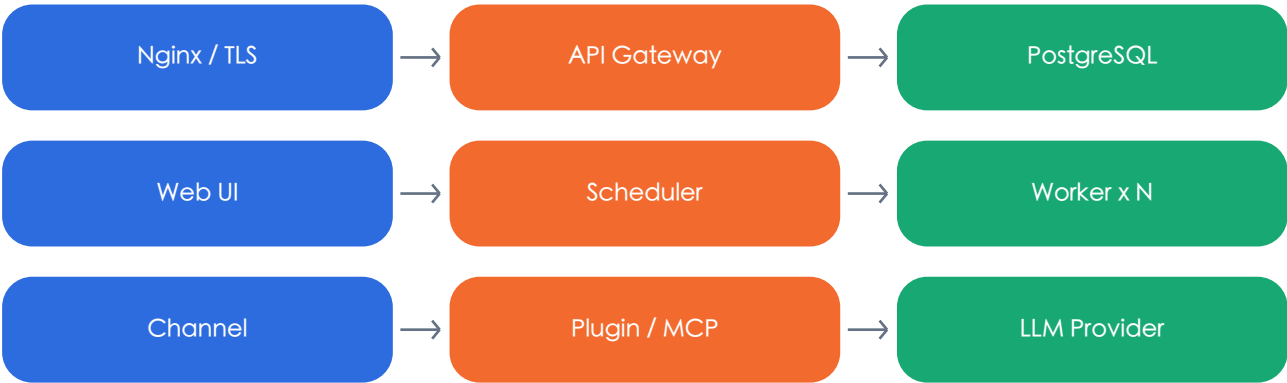
PostgreSQL 既是持久化存储，也是多个 API、Scheduler、Worker 进程之间的协调面。系统不依赖单机内存作为任务事实来源。

机制	作用	解决的问题
事务	原子写入 Run、Task、事件和状态	部分写入和状态漂移
SKIP LOCKED	并发领取任务的唯一裁决	多个 Worker 重复执行
Lease / Heartbeat	标识执行所有权并支持过期恢复	Worker 崩溃后任务悬挂
LISTEN / NOTIFY	低延迟唤醒调度器和 Worker	固定轮询带来的延迟
Outbox	保证外部消息投递可重试	网络失败导致结果丢失
Advisory Lock	迁移和共享初始化的互斥	多进程重复建表

扩展方式

增加 API 进程、Scheduler 或 Worker 不改变数据模型；Worker 通过 PG 竞争任务，Channel 通过 Lease 竞争连接所有权，所有节点都能从同一状态恢复。瓶颈主要来自数据库连接池、模型供应商限流、工具外部服务和单个 Agent 的并发上限。

10 部署与运行拓扑



当前线上形态

- 域名：dinq.smartjob.top，公网入口由反向代理转发至本机 API。
- API、Scheduler、Worker 使用独立 systemd 服务，工作目录为 /opt/joyhousebot。
- 配置通过 config.json 和环境文件提供；运行时状态在已有 PostgreSQL 中。
- 前端构建产物发布到 joyhousebot/static/ui，由 API 或反向代理提供。
- 可以在同一台服务器运行多个 Worker，也可以扩展到多节点，只要连接同一 PostgreSQL。

启动与健康检查

服务启动后检查 /healthz 和 /readyz；控制台顶部同时展示 API / PostgreSQL 状态。部署前应备份代码、配置和静态资源，重启后验证 systemd 状态、端口、数据库连接和公网页面。

安全注意

当前开发部署仍可能启用 allow_insecure_auth=true。生产环境必须关闭该选项，使用数据库 API Token、管理员表和最小权限；Channel Secret、LLM Key 和数据库连接串只能来自安全环境变量或密钥管理系统。

11 当前优势、边界与演进方向

方面	优势	当前边界 / 下一步
并发	PG 事务、SKIP LOCKED、Lease 支持多 Worker。	需根据连接池、模型限流和工具耗时做压测与容量配置。
可恢复	Run、Task、事件、记忆和 Outbox 都可持久化。	需持续验证异常重启、超时、死锁和重复投递场景。
可解释	事件时间线和 Trace 能定位每一步瓶颈。	需完善敏感信息脱敏、Trace 保留策略和权限模型。
业务隔离	dinq-plugin 可独立演进，核心只提供运行能力。	需要稳定 SDK、插件版本兼容和契约测试。
记忆	多层、可配置、用户和 Agent 隔离、PG 持久化。	需补充检索质量评估、冲突合并和隐私生命周期。
入口	HTTP、UI、Channel 统一进入 Run。	需完善各 Channel 的生产级限流、重连和监控。

推荐演进顺序

- 先完成 API Token、管理员权限、密钥保护和生产安全基线。
- 以 dinq-plugin 建立工具、Skill、Scenario 的契约测试和回放样例。
- 增加 PG NOTIFY 延迟、队列深度、Worker 利用率、LLM 成本和失败率指标。
- 建立压测矩阵：单用户长任务、多用户短任务、工具慢响应、Worker 重启和数据库故障。
- 再引入可选缓存、模型路由和跨节点弹性伸缩，不改变统一 Run 契约。

结论

Joyhousebot 当前最重要的架构特征是：以 PostgreSQL 为共享状态中心，以 Coordinator 为统一协调入口，以 Worker 集群执行持久化任务，以 Plugin 隔离业务能力，以事件和 Trace 提供全过程可解释性，并通过 Channel、Memory 和 Web 控制台扩展服务边界。

附录 A 关键概念速查

概念	一句话定义
Run	用户一次请求对应的可持久化执行实例。
Task	Run 中可调度、可重试、可并发的执行单元。
Coordinator	负责意图、场景、追问和执行计划的主 Agent。
Scenario	业务人员可配置的路由、字段、追问和能力边界。
Skill	可复用的策略和工具组合。
Tool	具有明确输入输出契约的一次外部操作。
Plugin	独立承载业务能力的扩展包，例如 dinq-plugin。
Channel	外部消息平台的接入和结果投递适配器。
Memory	按用户和 Agent 隔离的可配置持久化记忆。
Trace	模型、工具、阶段和结果的可回放诊断记录。
Lease	任务或 Channel 所有权的带过期时间租约。

附录 B 代码导航

API : joyhousebot/api/ ; 运行时 : joyhousebot/runtime/ ; PostgreSQL : joyhousebot/storage/ ; 记忆 : joyhousebot/agent/memory*.py ; Channel : joyhousebot/channels/ ; 前端 : frontend/ ; 业务插件 : 独立 dinq-plugin 项目。

本文档描述当前实现基线。代码演进后，应同步更新本文件并重新执行部署和验证流程。